CHAPTER 1

Introduction to Python

Abstract: Python is an object-oriented programming language that can support a wide range of applications like web development, desktop applications, *etc*. Its general-purpose programming features make coding easy, comprehensive and readable even for beginners.

Keywords: Development environments, Object-oriented programming, Operating system, Shuffle exchange network, Virtual machine.

INTRODUCTION

Python has grown to be a mainstream language. Its simple syntax and comprehensible code make it a popular general-purpose programming language among developers, engineers, and amateurs with limited programming skills. Its open-source features with a wide variety of libraries facilitate efficient programming. The first chapter of this book provides a brief introduction to Python along with a stepwise guide to installation on Windows and Linux. It will familiarize you with the Python IDE and editors, thus paving your journey to Python programming.

Guido van Rossum, Python's Benevolent Dictator for Life, developed Python in the 1990s. After that, several versions of Python were released. With the end of life of version 2.7, currently, versions 3.6 and 3.9 are widely used. Python is an open-source project maintained by the Python Software Foundation. We can take a tour of the Python universe at www.python.org.

Python is an object-oriented language that, due to its interoperability with existing codes in C and Fortran, gears up to the developer's demands and enhances the programmer's productivity while cutting down the time consumed. Python has become the developer's choice in various fields. For example, some of the prominent areas where Python is used are NASA's Jet Propulsion Laboratory, the Lawrence Livermore National Laboratory, Shell Research Boeing, Industrial Light and Magic, Sony Entertainment, and Procter & Gamble. Python is a high-level language; thus, the programs coded in it are easy and comprehensible. Python codes execute on a virtual machine; hence, a layer of abstraction exists

Krishna Kumar Mohbey & Malika Acharya

between the code and the executing platform. Thus, Python is an interpreted language and produces portable codes that are cross-platform executable. Fig. (**1**) depicts the execution of a simple Python program. Unlike Fortran, Python is a dynamically typed language that uses an interpreter to interpret the representative types at the run time. The layer of abstraction abstracts the underlying optimization from the code. Python binds Fortran and C libraries using an interpreter to perform intensive computation.

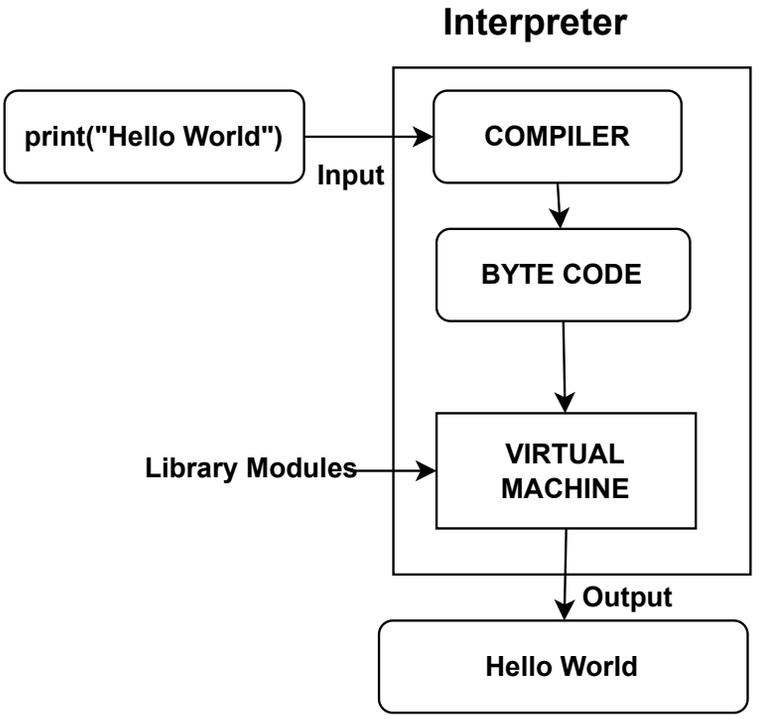

Fig. (1). Execution schematic of Python code.

You must have previously understood various languages like C++, Java, Perl, Scheme, or BASIC. All these languages are high-level languages. Nevertheless, computer hardware understands only low-level language called machine language. So, high-level language needs to be converted into machine language. For this, we have two translators: a *compiler* and an *interpreter*. A compiler is a program that translates the entire program into machine language. The high-level program is known as the source code, and the output is machine code. An interpreter is also a program that takes high-level language and converts it into machine code instruction by instruction. Compiled code can be run any number of times without repeated compiling or the source code.

In contrast, interpreted code requires an interpreter and source code every time. Thus, interpreted code makes the programming environment more user-friendly and enhances portability. Every CPU has its machine code, but we can run a high-level language program with the interpreter.

TECHNICAL STRENGTHS OF PYTHON

Portability

As already discussed, Python is a portable language that requires Byte Code for execution on any platform. An executable byte code converts the source code to platform-independent code. Python consists of a Tkinter toolkit to support the Tk GUI interface so the graphical user interface can run on all GUI's supported platforms without program changes. Python's original, standard implementation was given in ANSI C, making it executable on all major platforms ranging from PDAs to supercomputers.

Object-Oriented

Python is an object-oriented programming language. The language is extensible, *i.e*., the programs can be extended to C, C++, and Java. It uses a class model to support polymorphism, operator overloading, and other notions. It is a powerful scripting tool for other object-oriented system languages, like C++, Java, and C#. The recent versions of Python also support functional programming. This includes generators, comprehensions, closures, maps, *etc*.

Community Support

Python community support is quite active and responsive to the user's queries. The community consists of Python creator Guido van Rossum, the Benevolent Dictator for Life (BDFL), and a crew of thousands of workers. Python is more conservative than other languages in terms of changes, *i.e*., the changes need to be approved by the community, especially BDFL.

Advanced Features

Python provides full support for features of scripting languages like PERL. The scheme facilitates the use of software development tools that are easily found in compiled languages. It is a product of the FLOSS community, *i.e*., Python is a Free, Libre, and Open-Source Software that assists in knowledge sharing. Some of the salient features of Python are:

Memory Management

Memory allocation and reclamation are no longer the programmer's headache. It adopts automatic memory management allocation and garbage collection so that memory growth and shrink are as per demand. Python considers memory as the private heap. There are two types of allocators for memory management: raw memory allocator and object-specific allocator:

- **Large system development**: Python, with its vast repository of tools, allows faster and easier development of extensive systems. Python tools can be classified into two categories: general-purpose and functional tools. The generic tools include modules, classes, and exceptions that help to disintegrate large applications into small modules, reuse customized codes, and handle errors and exceptions.
- **Dynamic typing**: The programmer can easily write code without worrying about the program's syntax and structure. There is no need for the type and variable declaration beforehand.
- **Vast Library and Tools**: Python has full-fledged support for various built-in objects types like lists, dictionaries, *etc*. Python's standard operations, like concatenation, slicing, sorting, *etc*., allow sequential processing of these object types. Further, Python has some precoded library tools to support application-level specific tasks like networking, regular expressions, *etc*. It also has an extensive library to handle threading, GUI, *etc*. The easy integration of Python allows customization and communication with other frameworks. For example, integration of Java and .NET components allows easy communication with frameworks like COM and Silverlight.

Ease of Use

Python has become the developer's first choice due to its interactive user interface and rapid turnaround time. Python is quite easy to learn with its simple and flexible programming paradigm. Python majorly emphasizes on quality and comprehensibility of code rather than its structure. The absence of lengthy compilation time and immediate execution boosts the developing time.

Installing Python

Visit the official website www.python.org and get the latest version of Python. Fig. (**2**) shows the webpage. If the operating system is Mac or Linux, Python is already installed, although it might be an older version.

Fig. (2). Selection of Python for the operating system.

Windows Installer

1. Once you visit the Python official website, it will prompt you to download the latest version of Python for the respective operating system. You can select the respective operating system and it will take you to the available versions of Python for that operating system.

2. Select a Python version depending on your configuration (32-bit or 64-bit) from the list of options and it will start the automatic download of the executable file.

3. Once the download is complete, go to the downloading site and run the executable file.

4. A window will pop up that shows the default installation destination and also provides the option to customize the installation destination by browsing the same. Further, it prompts two checkboxes to *Add a Python Version to PATH* and to use the administrator privileges. This will confirm that the interpreter is added to the execution path.

5. Click ***Install*** Now. You will be prompted to *Optional Features* . Select the features as required.

6. Click ***Next*** . You will be prompted to the ***Advanced options*** features. Select the features as required. Also, here you will be required to choose the installation location.

7. Click ***Install*** . Your setup progress bar will show the progress of the installation. After the installation is complete a window will prompt the successful installation.

8. Once the installation is complete, you can open it by searching in the taskbar.

Ubuntu

With different versions of the Ubuntu distribution, Python is preinstalled. You might require updating the version of Python. The steps for Python installation on Ubuntu are as follows:

1. Check the version of Python installed on your Ubuntu system.

```
python -version
```

2. Update and refresh the list of the repository.

```
sudo apt update
```

3. Installation of Supporting Software.

```
sudo apt install software-properties-common
```

4. Add Deadsnakes PPA ((Personal Package Archive)).

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

5. You will be prompted to press enter for installation continuation. Press Enter. Refresh the package lists again.

```
sudo apt update
```

6. Install Python 3 using the following command.

```
sudo apt install python3.9
```

7. Verify the Python version after successful installation.

Linux Mint

Installation of Mint is like Ubuntu, and installation instructions for Ubuntu can also be used in Mint. A Deadsnakes/PPA working well with Mint.

Python IDLE

IDLE is the acronym for Interactive Development and Learning Environment. This IDLE helps to run your programs. Python has an IDLE for Windows that provides an interactive platform for program development. IDLE for Python comes in two modes: interactive mode and script mode. IDLE starts in the shell; you can type your code and see the output on the same window. Fig. (**3**). depicts the Python shell. You can search the Python shell in the Windows search bar.

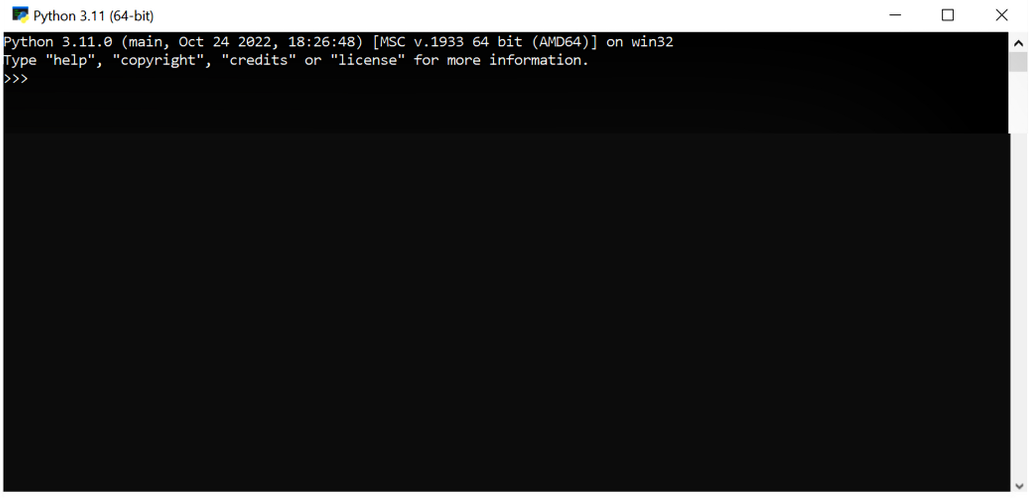

Fig. (3). Python Shell.

Anaconda Open-source Distribution

Anaconda, a Python package manager, provides applications suitable for large data processing, scientific computing, *etc*. It works for both Python and R programming languages. It provides various development platforms like JupyterLab, Jupyter, Notebook, Spyder, Glue, Orange, RStudio, and Visual Studio Code.

Installing Anaconda on Windows

1. Visit the official website of Anaconda: https://www.anaconda.com/distribution/#download . Fig. (**4**) depicts the site page. This link will prompt you to select the operating system on which you require Anaconda to be installed. Selecting the operating system would lead to the automatic downloading of the executable file.

2. Download the executable file. Run the installer on your computer.

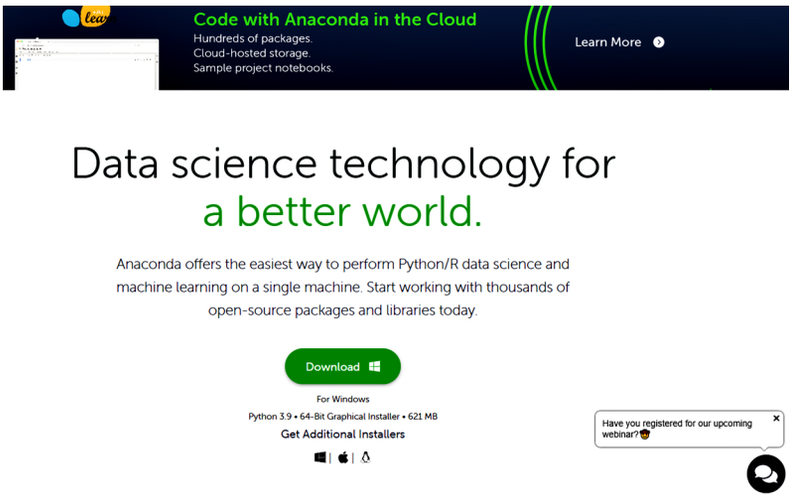

Fig. (4). Anaconda site page for Windows.

3. Once you select to download Anaconda, you will be prompted to agree to the license. Select the ***I Agree*** button to proceed.

4. You will be prompted to select ***Add path*** to the environment. Further, it will also prompt you to select where you want to install the Anaconda. Browse the installation destination and click ***Next*** .

5. In the next step select the installation type. In this, you get to select the user type for which the installation is to be done. It prompts two options, *Just me* or *All Users* . The former is used for the current user account and the latter is used for all users in the system, but it requires the users to consider administrator privileges. After selecting the type of installation, click on ***Next.***

6. Installation begins with the progress bar displaying the package being installed. In this window, there is a ***Show Details*** button from where you can check the progress of the installation along with the packages installed so far. After the installation is complete the window will display the " *Completed"* message.

7. Once the installation is complete, Anaconda provides two checkboxes one for starting the distribution tutorial and another to start with the Anaconda. Click on the ***Finish*** button to start the Anaconda shell.

8. You can open the Anaconda Navigator after the successful installation. In that Navigator, there will be several platforms available to start the coding. Over here we will focus on **Jupyter** and **Spyder** platforms.

Installing Anaconda on Linux

1. You can also download the file from the *curl* and *wget* retrieval commands.

2. Run the shell script using the bash command.

3. You will be prompted to select the destination folder and other settings during the installation. For Linux systems, keep the standard settings.

4. After installation, create an environment to install packages. You can also build an individual environment for each project.

First Python Program

In this section, we will write the first program using **Spyder**. Open Anaconda Navigator and select Spyder. Click on the "Launch" button present below the Spyder logo. Spyder IDE will open. Spyder IDE consists of three sections. The left pane is the workspace where you write code. The right pane is divided into two sections. The upper section is used for variables and object analysis. The lower pane is the console where the output occurs after successfully compiling the code. In IDE, you can write the following code in the left pane and execute it to see the result on the console.

Another very common application used for Python programming is **Jupyter**. Click on the launch button below the icon of Jupyter. It will prompt to the list of folders as depicted in Fig. (**5**). You can click on the "New" button to start the new Jupyter Notebook. The Jupyter Notebook is depicted in Fig. (**6**). You can type your code in the workspace and run each cell to get the output.

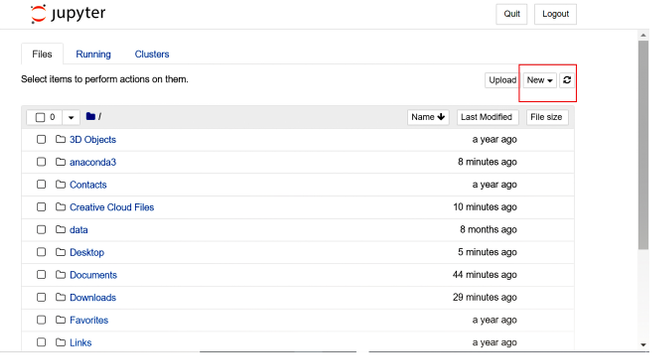

Fig. (5). Jupyter Home.

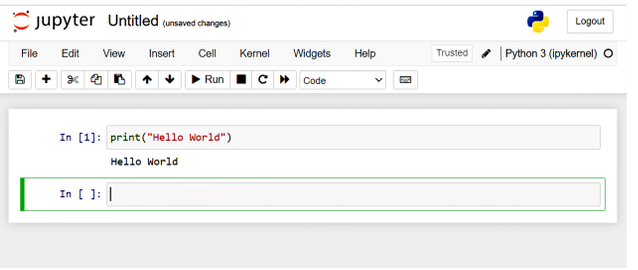

Fig. (6). Jupyter Notebook.

Example:

```
1. # Display a Name
2. print ("Hello World")
3. #Function displaying a name
```

Output:

```
Hello World
```

Python Keywords

Python keywords are words reserved for specific purposes and cannot be used for anything else. In total, Python has 35 keywords. The list of keywords has been revised many times. In Python 2.7, *print* and *exec* were keywords, but in Python 3+, they were eliminated, and in Python 3.7, *await* and *async* have been added as the new keywords. Below is the list of keywords for Python 3.9. You can use the help command to get the keywords in Python.

False	Def	if	Try	assert	except	nonlocal
Class	True	raise	In	else	lambda	yield
from	Pass	and	Elif	is	with	break
Or	global	del	As	while	await	for
None	continue	import	Return	async	finally	not

Identifiers

An identifier is a name that identifies a variable, function, class, object, *etc*. Some rules for identifier conventions are:

1. Do not use special characters like @, !, &, *, *etc*, and punctations in the identifier names.

2. An identifier should not start with a digit.

3. Identifiers are highly case-sensitive. Hence, ***atoms*** and ***Atoms*** are different identifiers in Python.

4. An identifier can start with alphabets *A to Z* and *a to z* or an underscore (_) that may or may not be followed by letters, underscores, or digits (0-9).

5. Do not use the keywords as the identifiers.

6. A class name must always begin with an uppercase alphabet.

7. A private identifier begins with a single underscore. A strongly private identifier begins with two underscores. Moreover, a special name ends with two trailing underscores.

y, dept, course_in_cs, course1, course_2, _grade, grade_2 ➔ **valid identifiers**
4, dep in college, cs-course, 1 st, 1_exam ➔ **Invalid identifiers**

Statements

In Python, a statement can be defined as an instruction or a command that the interpreter can execute. There are several kinds of statements in Python that offer a variety of purposes. We can compose a single line of multiple statements separated by semicolons. The various types of statements are:

1. Expression statement: They are used to compute, write a value or call a procedure.

2. Assignment statements: They assign the values and modify the items.

3. Assert statements: They provide debugging assertions.

4. Pass statements: They are much like null operations; no operation takes place when they are executed.

5. Del statements: As the name suggests, they specify the deletion targets.

6. Return statements: They are specified after the function block and defined when the program leaves the current function block.

7. Yield statements: They are used in the *generator* function only.

8. Raise statements: They raise the exceptions. If there exist no new exceptions, then they re-raise the exception that is currently being handled.

9. Break statements: They are usually used with loops (*for, while*) and with try statements. They end the execution sequence and move the program out of the current block in the loop.

10. Continue statements: They stop the current iteration in the loop and begin with the next iteration.

11. Import statements: They are used to import the modules that are required for easy pre-processing.

12. Global statements: They are used to define the identifier that remains valid for the entire code block.

13. Nonlocal statements: They bind the identifier to the nearest block so that searching for the identifier in the local namespace is very easy.

Indentation

In Python, indentation plays a very important role. Indentation can be defined as the blank spaces before a few statements. Besides enhancing the code readability, indentation also defines the scope of the function, much like the curly braces ({ }) in C and C++. Fig. (**7**) depicts the indentation for the loop. The indentation rules are as follows:

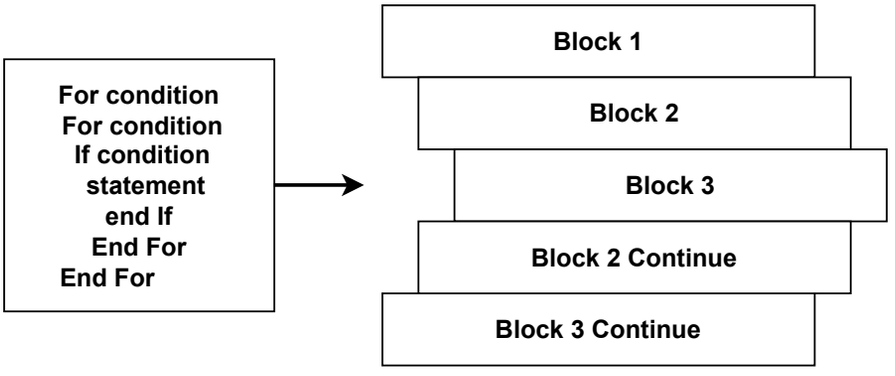

Fig. (7). Indentation in Python.

1. “Four spaces” are the default indentation spaces.

2. There cannot be an indentation in the first line of the code.

3. Maintain the uniformity of code, *i.e.*, the number of spaces must be uniform.

4. Prefer the use of whitespaces rather than tabs for indentation.

Comments

Comments in Python are statements added in the code by the programmers that indicate the purpose of the code block. They increase the readability of the code. They are not executed at the run time. The various purposes of comments are:

1. Enhanced readability.

2. Inclusion of resources.

3. Reuse of the existing codes.

4. Easy comprehension of the code and handy explanation.

Comments start with a # symbol and can be placed at the end of a line. There are two types of comments in Python. These are:

Single-line comments and *Multiline comments.*

• **Single line comments**

It starts with a # symbol which means you have to write the # symbol in every line.

• **Multiline comments**

A multiline comment in Python is a block of text enclosed by a delimiter (“”””) at the beginning and end of the comment. There should be no white space in delimiters (“”””).

Let us take an example for understanding comments.

Example:

```
1.  print ("Python Programming") # Single line comment
2.  """
3.  This is a comment
4.  written in
5.  more lines
6.  """
7.  print ("Book for Computer Science.")
```

Output:

```
Python Programming

Book for Computer Science.
```

Coding Styles

Python offers four different coding styles, as discussed below.

Functional

In this style, the problem is divided into different functions, and each statement is considered a Mathematical equation. It is widely used in parallel processing tasks and lambda calculus. It is the first choice for large data processing jobs.

Example:

```
1. my_list = [2,3,6,7,8,10]
2. new_list = list(map(lambda x: x + 2, my_list))
3. print(new_list)
```

Output:

```
[4, 5, 8, 9, 10, 12]
```

Imperative

This style is used explicitly for various operations on data structures. It requires a pre-definition of the order of operations.

Example:

```
1.  my_list = [2,3,6,7,8,10]
2.  sum = 0
3.  for x in my_list:
4.  sum += 2*x
5.  print(sum)
```

Output:

```
72
```

Procedural

It is the most adopted style by nonprogrammers. It primarily focuses on the code. The entire code is divided into a list of instructions, like breaking a problem into simpler tasks. It uses sequencing, selection, modularization, and iteration to reduce code redundancy.

Example:

```
1. my_list = [2,3,6,9,18,20]
2. def add(list):
3. sum = 0
4. for x in the list:
5. sum += x
6. return sum
7. print(add(my_list))
```

Output:

```
36
```

Object-oriented

In the procedural style, the focus was on the given code, but in this style, the focus is on the data. The merit of this style is the formation of a class paradigm and adherence to object-oriented principles like abstraction, inheritance, polymorphism, and encapsulation.

Example:

```
1. class A:
2. def test(self,a,b):
3. return(a+b)
4. b=A()
5. print("res=", b.test(45,77))
```

Output:

```
res=122
```

CONCLUDING REMARKS

In this chapter, we furnished a general introduction to Python. This chapter also guided the readers through the installation of Python and Anaconda over different operating systems and for different configurations. We also familiarised the readers with some basic concepts of Python programming. In the next chapter, we will look deeply into Python data types and operators.